

Projekt

Wiktoria Grodzka

1 Wstępna obróbka danych

Zbiór danych składa się z 1500 obserwacji treningowych oraz 500 obserwacji testowych, opisanych przez 100 cech. Dane zostały wczytane z plików CSV, a etykiety klas przekształcono do postaci binarnej (klasa 2 zamieniona na 0), zgodnie z wymaganiami algorytmów GradientBoosting.

Zbiór treningowy został dodatkowo podzielony na część treningową i testową przy `test_size=0.15` oraz `random_state=42`. Zastosowałam strategię walidacji krzyżowej `StratifiedKfold(n_splits=5, shuffle=True, random_state=42)`, która zapewnia zachowanie proporcji klas w każdym foldzie.

2 Ogólna metodologia

Proces modelowania oparty został na pipeline'ach łączących przetwarzanie danych, selekcję cech oraz klasyfikację. Strategia poszukiwania parametrów została podzielona na dwa etapy: najpierw identyfikację konfiguracji o wysokim potencjale predykcyjnym (`GridSearchCV`), a następnie świadome ograniczanie overfittingu poprzez regularyzację.

3 Wybór skalera

Na początkowym etapie analizowałam wpływ skalowania danych, testując *StandardScaler*, *RobustScaler* oraz brak skalowania. Wstępne eksperymenty wskazywały na przewagę *RobustScaler*, który został zastosowany w większości pipeline'ów.

4 Selekcja cech

Testowałam następujące metody selekcji cech:

- *SelectKBest* z miarą `mutual_info`, w której najlepiej sprawdziło się `k=40`
- *RFE* z *GradientBoostingClassifier*
- *SelectFromModel* z estymatorami GradientBoosting czy RandomForest
- *VarianceThreshold*, który przy `threshold=0.01` nie wykazał wpływu

Dodatkowo sprawdziłam czy dodanie syntetycznych danych za pomocą SMOT okaże się przydatne, jednak nie okazało się korzystne. Ostatecznie najlepszym selektorem okazał się *SelectFromModel* wykorzystujący *GradientBoostingClassifier* z pogiem `threshold="1.2*mean"`.

5 Fine-tuning z wykorzystaniem GridSearchCV

Choć `GridSearchCV` nie wydaje się najlepszą metodą uzyskania wyższego BA na zbiorze testowym, postanowiłam się nim posłużyć by zawęzić pole poszukiwań odpowiednich parametrów, a następnie minimalizować overfitting. Na tym etapie zastosowałam strategię mikro-dostrajania hiperparametrów poprzez przeszukiwanie wąskiego otoczenia dotychczasowych najlepszych konfiguracji. Zdecydowałam się skupić na optymalizacji parametrów w GradientBoosting, ponieważ wyniki reszty modeli nie były satysfakcjonujące.

5.1 Optymalizacja GradientBoosting

Optymalizacja `n_estimators` (400-600), `max_depth` (6-8), `subsample` (0.8-0.9). Etapowe dostrajanie wokół najlepszych parametrów finalnie wskazało:

```
n_estimators=600 learning_rate=0.07 max_depth=6 min_samples_split=5 min_samples_leaf=1
subsample=0.8
```

5.2 Wszystkie analizowane modele

Dodatkowo optymalizowałam również następujące modele: `GradientBoostingClassifier`, `VotingClassifier(XGBoost, GradientBoosting, RandomForest)`, `HistGradientBoostingClassifier`, `VotingClassifier` względem `random_state` w GB i SFM(GB), które finalnie zwróciły odpowiednio:

Model	BA Test	BA Train	CV mean	CV std
HGB	0.738	1.000	0.751	0.031
V(XGB,GB,RF)	0.733	1.000	0.757	0.027
GB_best	0.826	1.000	0.839	0.016
V(GB_best(random_seed))	0.835	1.000	0.847	0.014

6 XGBoost i LightGBM

Kolejnym krokiem było przetestowanie bardziej zaawansowanych wariantów GB: **XGBoost** oraz **LightGBM**. XGBoost wprowadza regularyzację L1 i L2, co pozwala na lepszą kontrolę nad overfittingiem. LightGBM z kolei stosuje strategię wzrostu drzew w kontekście liści zamiast poziomów, co prowadzi do bardziej efektywnego wykorzystania dostępnych zasobów obliczeniowych przy jednoczesnym osiąganiu lepszej dokładności. Ostateczne wyniki pierwszego etapu optymalizacji to:

Wyniki modeli przed regularyzacją overfittingu

Model	Parametry wpływające na overfitting	BA test	BA train	CV mean	CV std
XGBoost	<code>n_estimators=400</code> , <code>learning_rate=0.07</code> , <code>max_depth=6</code> , znikoma regularyzacja	0.860	1.000	0.853	0.011
LightGBM	<code>num_leaves=40</code> , <code>min_child_samples=5</code> , brak regularyzacji	0.858	1.000	0.849	0.022

7 Kontrola overfittingu

Overfitting był głównym problemem w zaawansowanych modelach, które osiągały `BA train` = 1.0 przy `BA test` nie większym niż 0.85. Najważniejsze parametry ograniczające przeuczenie, na których po kolei się skupiałam w tym etapie to:

`max_depth` (im mniejsze tym płytsze drzewa, mniej skomplikowane), `subsample` (używanie tylko losowej części danych do budowy każdego drzewa zapobiega zapamiętywaniu konkretnych obserwacji treningowych), `colsample_bytree` (ogranicza liczbę cech dostępnych dla każdego drzewa, zmuszając model do generalizacji), `min_child_samples` (im większa wartość tym mniej specyficzne reguły), `reg_alpha` i `reg_lambda` (regularyzacja L1 i L2, karanie złożoności modelu), `n_estimators` (zmniejszenie liczby drzew w modelu), `gamma` (minimalny zysk potrzebny do podziału węzła). *Tutaj LightGBM 1 ma dużo za wysokie, `n_estimators`.

Wyniki części modeli uczestniczących w fine-tuningu tego etapu.

Model	Kluczowe zmiany	BA test	BA train	CV mean	CV std
XGBoost 2	reg_alpha=0.2 n_estimators=300 reg_lambda=4	0.822	0.991	0.848	0.025
XGBoost 3	reg_alpha=0.3 n_estimators=320	0.849	0.989	0.860	0.016
XGBoost 4	colsample_bytree=0.85 subsample=0.8 gamma=0.2	0.858	0.991	0.854	0.005
LightGBM 1	reg_alpha=0.1 reg_lambda=1.0	0.871	1.000	0.856	0.012
LightGBM 2	n_estimators=320	0.858	0.998	0.859	0.016

8 Voting

W ostatnim etapie połączyłam ze sobą w VotingClassifier: dwa najlepsze modeli: 1) XGBoost i LightGBM z GradientBoostingiem w VotingClassifier z wagami [2,2,1]

2) XGBoost i LightGBM z bardziej stabilnym XGBoostem z wagami [2,1,2]

Wyniki najlepszych modeli

Stage	BA Test	BA Train	CV mean	CV std
XGB	0.858	0.991	0.854	0.005
LGBM	0.858	0.998	0.859	0.016
Voting 1	0.871	0.998	0.861	0.015
Voting 2	0.866	0.995	0.863	0.010

Patrząc na poniższe wykresy modelu XGBoost i Voting 2 widzimy, że overfitting nadal jest obecny. Voting 2 przyniosło nieznaczny wzrost Balanced Accuracy na zbiorze testowym w porównaniu do pojedynczych modeli przy zachowaniu akceptowalnej stabilności. XGBoost jest najstabilniejszy w walidacji krzyżowej, co sugeruje najlepszą generalizację na nieznanymi danych, jednak z niższym wynikiem na zbiorze testowym. astosowanie

Biorąc pod uwagę kompromis między wydajnością a stabilnością predykcji, wygeneruję prawdopodobieństwa modelem Voting 2 (na całości oryginalnych danych treningowych).

